# Emotion Detection & Learning Support Engine

## Project Description:

The **AI-Driven Emotion Detection & Personalized Learning Support System** is an intelligent platform designed to analyze a learner's emotional state from their text input and provide personalized academic and emotional support. The system uses Natural Language Processing (NLP) and deep learning models such as **BiLSTM** and **BERT** to classify emotions like happiness, sadness, confusion, frustration, or anxiety from user input. It also includes a rule-based keyword enhancement approach to improve prediction accuracy.

Once a user submits a learning-related query, the system processes the text, detects the underlying emotion, and generates a supportive and context-aware response. A generative AI module (Gemini API) is used to provide personalized guidance and learning suggestions based on the detected emotion. The main goal of this project is to create an intelligent learning assistant that understands student emotions and helps improve their learning experience by providing motivation, guidance, and stress-free support.

The AI-Driven Emotion Detection & Personalized Learning Support Platform is an end-to-end system that transforms a learner's free-text description of their study challenge into an emotion-aware, supportive response. By combining deep learning emotion classifiers (BiLSTM and BERT) with rule-based keyword enhancement and Gemini-generated guidance, the platform ingests raw student input, predicts nuanced emotional states (Bored, Confident, Confused, Curious, Frustrated), and delivers tailored tips, next steps, and encouragement. This automated workflow reduces manual triage, surfaces real-time emotional insight, and maintains consistent, empathetic responses within seconds. Beyond prediction, the system logs interactions to CSV for continuous learning, visualizes emotion trends through an analytics dashboard, and supports side-by-side model comparisons. Users can run the Streamlit app to enter their problem context, see mixed-emotion breakdowns, toggle AI responses, and review personalized strategies. This creates a fast, transparent loop that improves learner support quality while minimizing response time and effort.

## Scenarios

**Scenario 1:**

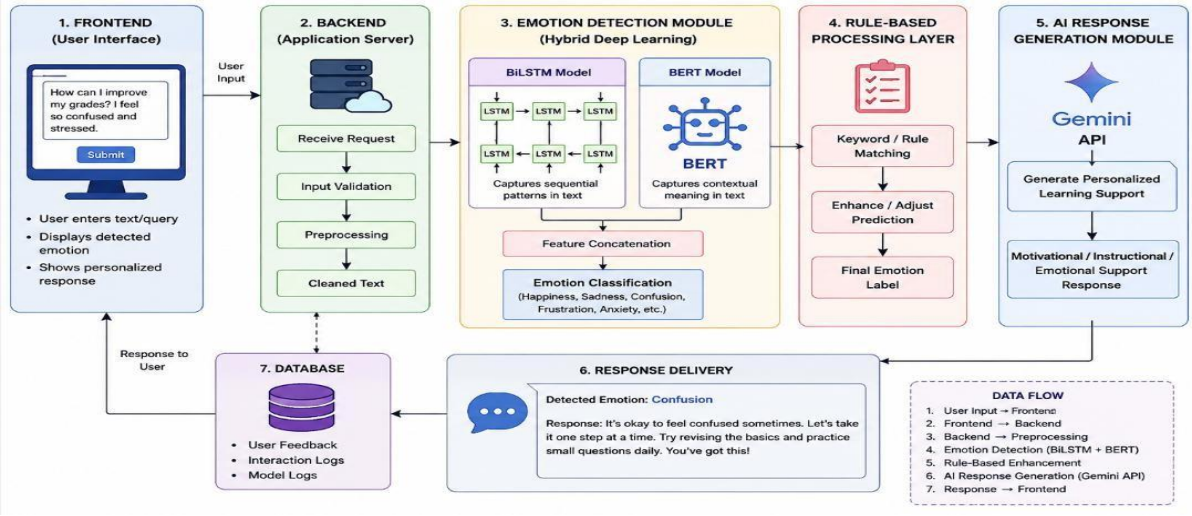
Educators, TAs & Academic Support Teams Instructors and academic support staff often spend significant time interpreting student sentiments from messages and emails before offering help. Manual reading, guessing emotions, and crafting the right tone slows response cycles and can miss struggling students. The AI Learning Assistant removes these bottlenecks by automatically classifying emotions from student text, highlighting confidence levels, and generating supportive, field- specific replies. A TA, for instance, can paste a student's concern and instantly see whether

they’re Confused or Frustrated, along with suggested guidance. Dual- model comparison (BiLSTM vs BERT) and mixed-emotion detection improve reliability and transparency. Teams accelerate triage, maintain consistent empathy, and focus more on targeted academic intervention rather than manual sentiment interpretation.

**Scenario 2:**

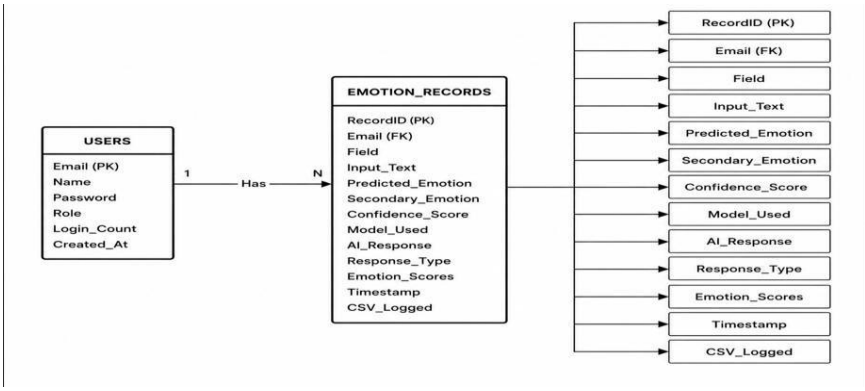
Learners & Self-Study Educators, trainers, and students frequently create lecture slides, learning materials, and project presentations Students working independently often need quick, empathetic guidance when stuck or unsure. Manually searching forums or drafting help requests takes time and may not capture their feelings. The Learning Assistant streamlines this by letting students describe their challenge in plain language and immediately receiving emotion-aware tips and next steps. For example, a learner can note “I’m lost on recursion,” and the system will flag Confused, surface clarity-focused advice, and provide encouraging direction. Mixed-emotion outputs (e.g., Curious + Confused) help learners self-reflect, while saved interactions build a history for progress tracking. This makes the platform a powerful companion for self-paced study and rapid clarification.

## Technical Architecture:




**Description:** The system follows a layered architecture consisting of a **frontend interface**, **backend API server**, **machine learning processing layer**, and a **data storage layer**. The user submits a text input through a web-based interface, which is sent to the backend built using Flask or FastAPI. The backend first routes the input to a preprocessing module where the text is cleaned, tokenized, and prepared for analysis. The processed input is then passed to the emotion detection models, including **BiLSTM and BERT-based classifiers**, which predict the user’s emotional state such as stress, confusion, or frustration. Simultaneously, a response generation module uses rule-based logic and the Gemini API to create personalized learning suggestions and emotional support feedback. Finally, the system returns both the detected emotion and supportive response to the frontend, while also storing logs and results in a database for future training and analysis.

## ER Diagram:




**Description:** The ER (Entity–Relationship) diagram of the AI-Driven Emotion Detection & Personalized Learning Support System represents the structure of how data is stored and related within the system. The main entities in the system include **User (Student)**, **Input Text**, **Emotion**, **Prediction Model**, **Learning Response**, **Feedback**, and **Admin**. The User entity stores student details such as user ID, name, and login credentials, and it submits Input Text describing learning difficulties. Each Input Text is analyzed by the Prediction Model, which identifies the corresponding Emotion such as happy, sad, stressed, or confused. Based on the detected Emotion, the system generates a personalized Learning Response that provides guidance, motivation, or study support. The Feedback entity captures user responses to the suggestions provided, helping to improve system accuracy and effectiveness over time. The Admin entity manages datasets, model updates, and system monitoring. The relationships among these entities ensure that each user input is processed, analyzed, and mapped to an appropriate emotional state, which is then used to deliver personalized educational support and store feedback for continuous improvement of the system.

## Pre-requisites:

**1. Python Environment Setup**

Install Python 3.9+ and create a dedicated virtual environment for clean dependency management.

Official Download: https://www.python.org/downloads/

**2. Install Required Libraries**

All dependencies used in text extraction, LLM integration, Stable Diffusion image generation, PPT building, and the Streamlit UI must be installed from requirements.txt.

**3. Streamlit Installation**

Needed to run the interactive learning assistant UI.

Docs: https://docs.streamlit.io/library/get-started/installation

**4.Model assets and data**

Ensure model files exist under models/bltsm/ (BiLSTM) and models/bert_emotion_model_final/ (BERT weights/config/tokenizer). Datasets already sit in data/; no extra conversion tools are required for this app.

**5. Gemini API key**

Required for AI-generated supportive responses. Set GOOGLE_API_KEY (or update app.py to read from env instead of the hardcoded key). Gemini API Setup: https://aistudio.google.com/

**6. Optional Tools for Development**

Recommended IDEs for development and debugging:

- Visual Studio Code: https://code.visualstudio.com/
- PyCharm Community: https://www.jetbrains.com/pycharm/

These tools support linting, Python virtual environments, and structured project navigation

## Project Workflow:

**Epic 1. Environment Setup and Dependency Configuration**

Story 1: Obtain Gemini API Key
Story 2: Install Python & Create Virtual Environment
Story 3: Install Project Dependencies
Story 4: Create the .Env Configuration File
Story 5: Verify Model and Data Directories
Story 6: Prepare Project Folder Structure

**Epic 2. Kaggle Model Training and Integration**

Story 1: Kaggle Setup (GPU + Dependencies + Data Loading)
Story 2: Data Preprocessing & Tokenization
Story 3: BiLSTM Model Training
Story 4: Domain-Adaptive Fine-Tuning
Story 5: BERT Model Fine-Tuning
Story 6: Model Export & Local Integration

**Epic 3. Core Emotion Detection Pipeline Development**

Story 1: Text Preprocessing & Keyword Enhancement
Story 2: BiLSTM Classifier (5-Class Softmax)
Story 3: BERT Classifier with Class Weighting & Keyword Adjustments
Story 4: Mixed Emotion Detection (≥15% Secondary Scores)
Story 5: Unified Prediction Schema
Story 6: CSV Persistence & Cached Model Loading

**Epic 4. AI-Powered Guidance and Regeneration Engine**

Story 1: Capture Field and Problem Context for Prompt Generation
Story 2: Generate Empathetic and Context-Aware AI Responses
Story 3: Response Regeneration and Synchronization Mechanism
Story 4: Session History Management and CSV Logging

**Epic 5. Streamlit UI Implementation**
Story 1: Responsive Layout and Session State Management
Story 2: Emotion Analysis, Model Comparison, and Visualization Components
Story 3: Form Controls, Validation, and Error Handling
Story 4: Analytics Dashboard and Interactive Charts

**Epic 6. User Interaction**
Story 1: Validate UI Flow End-to-End
Story 2: Optimization and Deployment Readiness

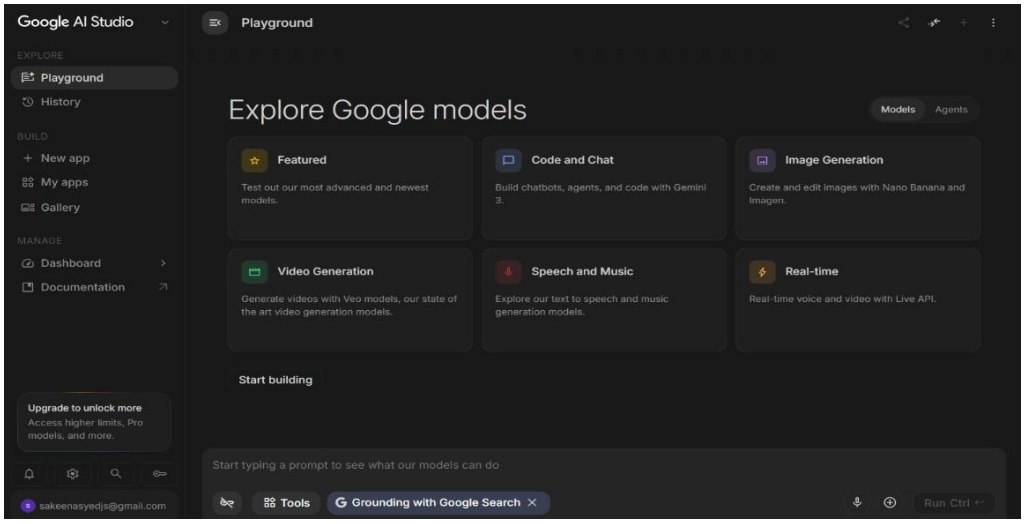
**Conclusion**

# Epic 1: Environment Setup and Dependency Configuration

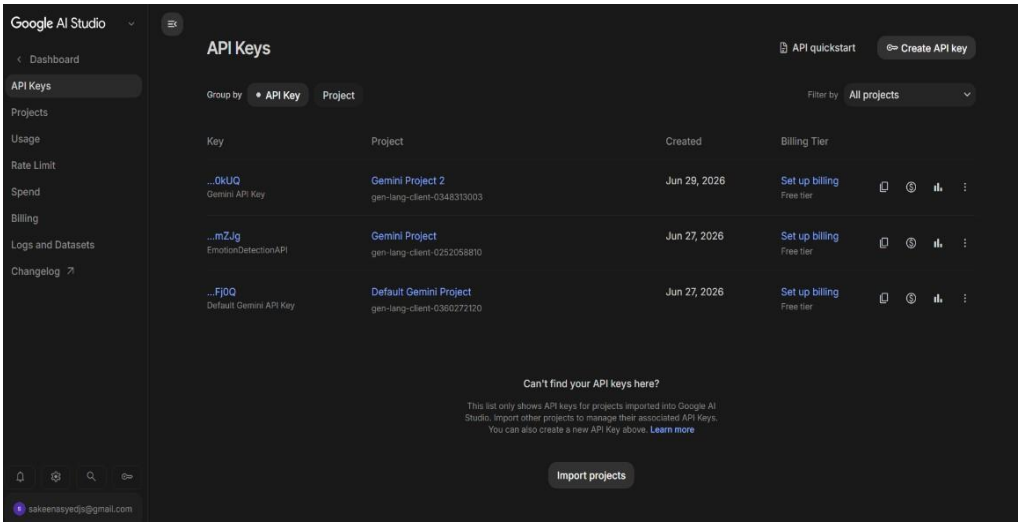
**Description:** Epic 1 focuses on setting up the complete development environment required for the Emotion Detection and Learning Support system. It includes installing Python, configuring a virtual environment, setting up project dependencies, and integrating external services like the Gemini API. This epic ensures that the system has all required tools, libraries, and configurations before model training and development begin. A proper folder structure is also created to maintain clean and organized project architecture.

## Story 1: Obtain Gemini API Key:

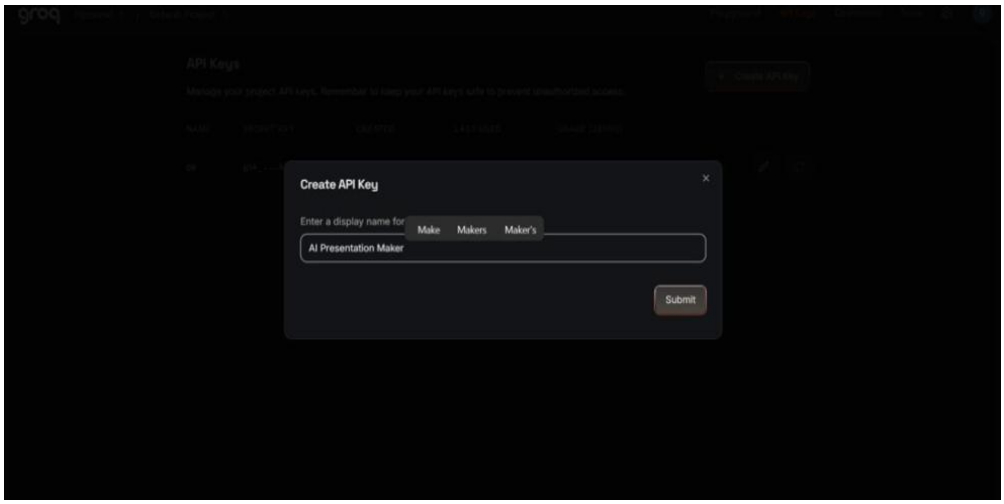
- Visit the official Google AI Studio: https://aistudio.google.com/
- Sign in with your Google account to access the Gemini models.



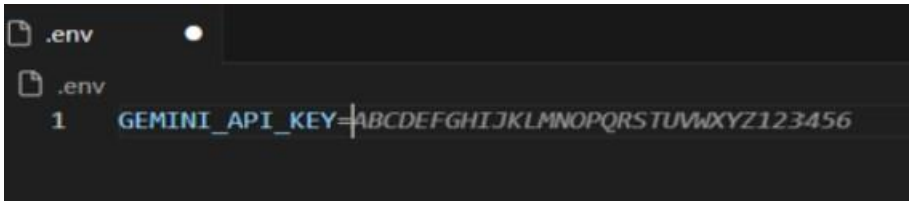

- Click on Get API Key / Create API Key, then generate a key for your project.




- Navigate to the API Keys section in the dashboard.
- Click Create API Key, then assign a recognizable name such as Emotion.




- Copy the generated key and store it securely in your project's .env file.




## Story 2: Install Python & Create Virtual Environment

- Install **Python 3.9+** from the official website: https://www.python.org/downloads/
- Create and activate a virtual environment:

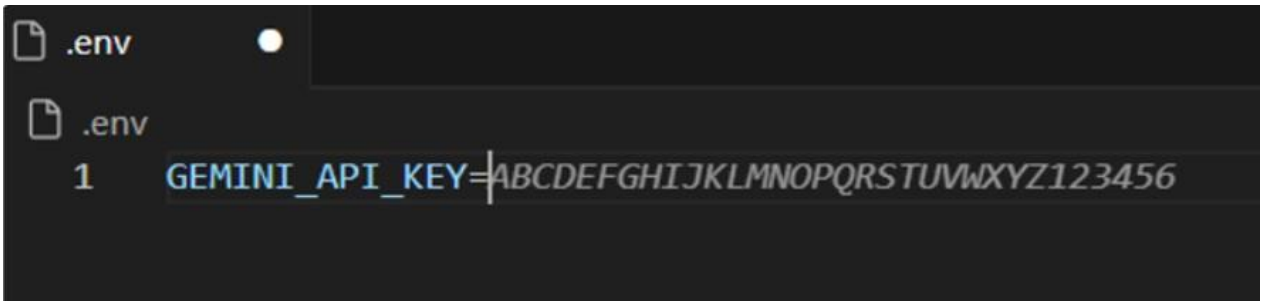

## Story 3: Install Project Dependencies

Use the existing requirements.txt file in the project root to install all required libraries for:

- **Emotion modeling** (TensorFlow/Keras BiLSTM, PyTorch/BERT)
- **NLP** (NLTK)
- **Data handling & analytics** (pandas, numpy, plotly)
- **Web UI & AI integration** (Streamlit, google-generativeai)

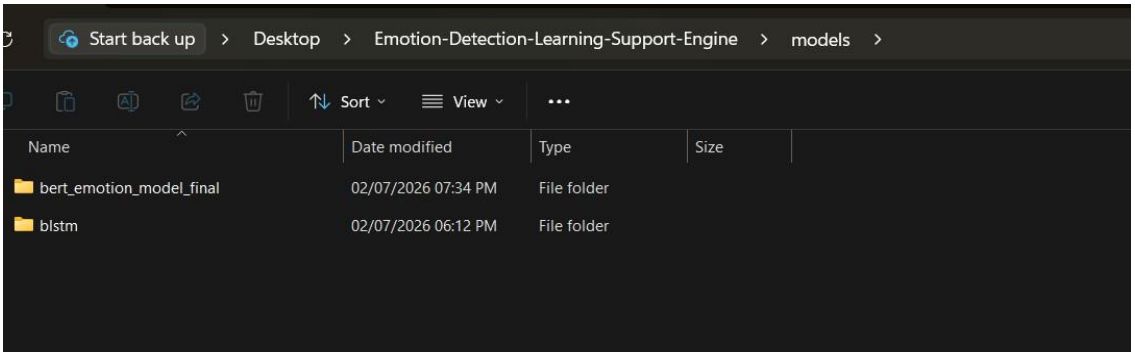

## Story 4: Create the .env Configuration File

Add the required environment variables, including:

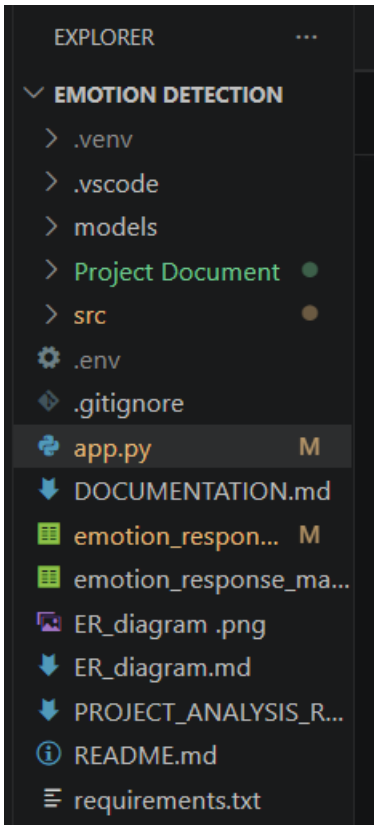



## Story 5: Verify Model and Data Directories

- Ensure that the trained model folders exported from Kaggle are placed correctly:
- models\bltsm\ (BiLSTM model, tokenizer, label classes)
- models\bert_emotion_model_final\ (BERT model, tokenizer, label mappings)
- Confirm that dataset folders under data\ (e.g., emotion_text_dataset.csv, GoEmotions, empatheticdialogues) are present.




## Story 6: Prepare Project Folder Structure



# Epic 2. Kaggle Model Training and Integration

It focuses on preparing, training, and integrating the BiLSTM and BERT emotion detection models.

It includes data preprocessing, tokenization, model fine-tuning, performance evaluation, and exporting trained models for deployment within the application.

## Story 1: Kaggle Setup (GPU + Dependencies + Data Loading)

The system processes raw student input text through cleaning, tokenization, and keyword enhancement before passing it to the emotion classification models. Each text undergoes specialized preprocessing to preserve emotional context while normalizing format.

● **Environment Configuration**: Utilized Kaggle's dual-GPU environment to accelerate deep learning workflows.

● **Dependency Management**: Installed specific versions of TensorFlow (2.17.0), NumPy (1.26.4), and Hugging Face Transformers to ensure cross-environment stability.

● **Data Verification**: Validated the presence of five core datasets, including **GoEmotions, EmpatheticDialogues**, and **ISEAR,** ensuring all CSV paths were correctly mapped.

```
print("📦 Installing Kaggle-safe compatible versions...")
subprocess.run([sys.executable, "-m", "pip", "install", "-q",
                "tensorflow==2.17.0",
                "numpy==1.26.4",
                "pandas==2.2.2",
                "scikit-learn==1.3.2",  # ✅ FIXED: Works with numpy 1.26.4
                "scipy==1.11.4",        # ✅ FIXED: Works with numpy 1.26.4
                "matplotlib==3.8.3",
                "seaborn==0.13.0",
                "nltk==3.8.1",
                "transformers==4.35.2",  # ✅ Simpler, fewer deps
                "torch==2.1.2",
                "plotly==5.18.0",
                "protobuf==4.24.4"], capture_output=True)

print("✅ Verifying installation...")
import tensorflow as tf
import numpy as np
import sklearn
import transformers

print(f"✅ TensorFlow: {tf.__version__}")
print(f"✅ NumPy: {np.__version__}")
print(f"✅ scikit-learn: {sklearn.__version__}")
print(f"✅ transformers: {transformers.__version__}")

# GPU setup
gpus = tf.config.list_physical_devices('GPU')
print(f"\n🔥 GPUs detected: {len(gpus)}")
for gpu in gpus:
    tf.config.experimental.set_memory_growth(gpu, True)

print("\n🎉 Kaggle environment is STABLE - NO CONFLICTS!")
```

```
✅ TensorFlow: 2.17.0
✅ NumPy: 1.26.4
✅ scikit-learn: 1.3.2
✅ transformers: 4.35.2

🔥 GPUs detected: 2

🎉 Kaggle environment is STABLE - NO CONFLICTS!
```

## Story 2: Data Preprocessing & Tokenization

LLaMA transforms the extracted content into a structured slide blueprint.

***The model generates:***

● Unified Dataset Creation: Combined multiple sources into a single dataframe of 198,476 rows with five standardized emotion classes: Bored, Confident, Confused, Curious, and Frustrated.

- Text Cleaning: Implemented regex-based cleaning to normalize text while preserving emotion-carrying punctuation.
- Sequence Preparation: Created a Keras Tokenizer with a 30,000-word vocabulary. Applied padding and truncation to a fixed 80-token length for consistent BiLSTM input.

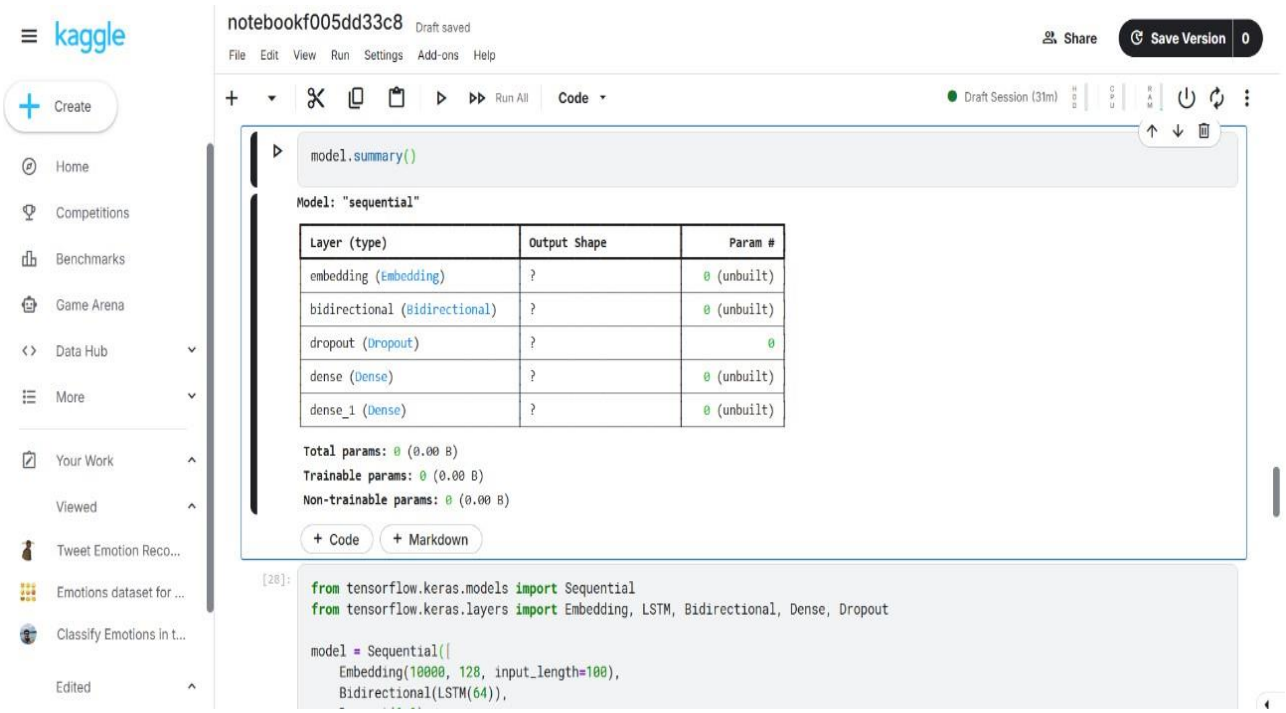
```
# ----------------------------
# Tokenization Function
# ----------------------------
def tokenize(batch):
    return tokenizer(
        batch["text"],
        padding="max_length",
        truncation=True,
        max_length=64
    )

train_dataset = train_dataset.map(tokenize, batched=True)
val_dataset = val_dataset.map(tokenize, batched=True)
test_dataset = test_dataset.map(tokenize, batched=True)

train_dataset.set_format(type="torch", columns=["input_ids", "attention_mask", "label"])
val_dataset.set_format(type="torch", columns=["input_ids", "attention_mask", "label"])
test_dataset.set_format(type="torch", columns=["input_ids", "attention_mask", "label"])

print("Tokenization done!")

# ----------------------------
# Model
# ----------------------------
model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=len(label_encoder.classes_)
)
```

## Story 3: BiLSTM Model Training

Slides containing visual content require Stable Diffusion images. Each slide's image prompt is optimized using LLaMA for clarity, realism, and subject relevance.

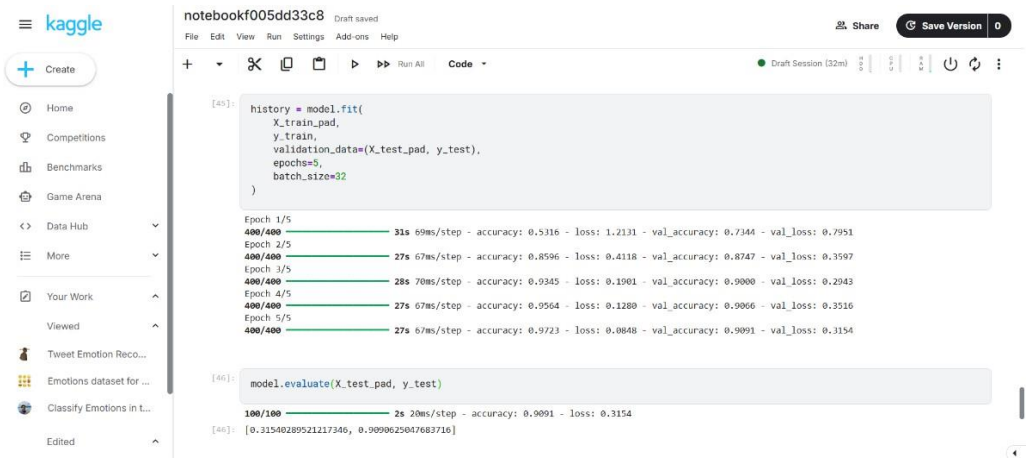
- **Architecture Design:** Developed a BiLSTM network with an Embedding layer (128-dim) and a Bidirectional LSTM layer (128 units), totaling **4.1 million parameters**.
- **Class Imbalance Handling:** Implemented **Focal Loss** ($\gamma=2.0$) and dataset balancing to ensure the model accurately identifies minority classes like *Bored* and *Frustrated*.
- **Performance Metrics:** Achieved a **95% overall accuracy**, with particularly high F1-scores for *Curious*
(1.00) and *Confused* (0.96).

```
# ----------------------------
# Model
# ----------------------------
model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=len(label_encoder.classes_)
)

# Move model to GPU if available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

print("Model loaded on:", device)
```



```python
# ----------------------------
# Trainer
# ----------------------------
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    compute_metrics=compute_metrics,
)

# ----------------------------
# Train
# ----------------------------
print("Starting Training...")

trainer.train()

print("Training Completed!")

# ----------------------------
# Save Model
# ----------------------------
trainer.save_model("models/bert_emotion_model_final")
tokenizer.save_pretrained("models/bert_emotion_model_final")

print("Model saved successfully!")
```

• Shows training progress with focal loss implementation, achieving high accuracy (93.97%) and low loss (0.0643) on training data, with validation accuracy of 83.39%. Training completes in 2.42 minutes with early stopping to prevent overfitting.



• Demonstrates model performance metrics with precision, recall, and f1-score for all 5 emotion classes. Shows high accuracy (95%) overall, with perfect performance on Curious class (1.00 precision/recall) and strong performance on Confused class (0.99 precision, 0.93 recall).

```
# ------------------------------
# Metric
# ------------------------------
accuracy = evaluate.load("accuracy")

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    return accuracy.compute(predictions=predictions, references=labels)
```

## Story 4: Domain-Adaptive Fine-Tuning

Using **python-pptx**, the system builds a polished PowerPoint file using predefined design templates.

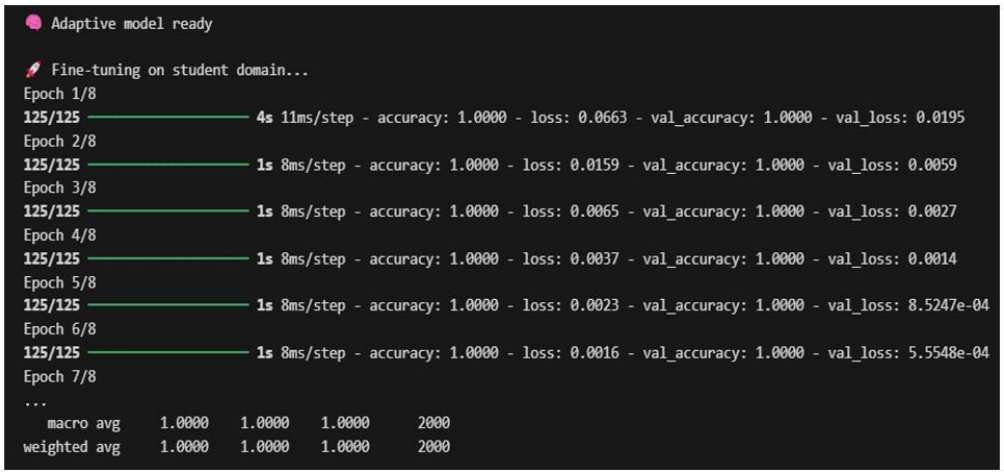
● **Targeted Learning**: Fine-tuned the baseline BiLSTM model on 10,000 student-specific samples to better capture academic-related emotional expressions.

● **Transfer Learning Success:** The model achieved 100% validation accuracy on the student dataset, successfully adapting to the specific vocabulary used by learners.

● **Artifact Generation**: Exported the final adaptive model as bilstm_student_adaptive.keras.

```
# ============================
# 5. CLONE MODEL FOR ADAPTATION
# ============================
adaptive_model = tf.keras.models.clone_model(baseline_model)
adaptive_model.set_weights(baseline_model.get_weights())

# Freeze embedding layer
adaptive_model.layers[0].trainable = False

adaptive_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

print("\n🧠 Adaptive model ready")
```

```
# ============================
# 6. FINE-TUNE
# ============================
callbacks = [
    tf.keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=3,
        restore_best_weights=True
    )
]

print("\n🚀 Fine-tuning on student domain...")

history = adaptive_model.fit(
    X_train,
    y_train,
    validation_data=(X_val, y_val),
    epochs=8,
    batch_size=64,
    callbacks=callbacks,
    verbose=1
)
```




## Story 5: BERT Model Fine-Tuning

- **Transformer Training:** Fine-tuned bert-base-uncased for three epochs using the **AdamW optimizer** with a learning rate of $2 \times 10^{-5}$.
- **Evaluation:** BERT demonstrated superior contextual understanding, maintaining a **95% accuracy** on the test set while showing stable loss reduction ($0.15 \rightarrow 0.05$).
- **Comprehensive Saving:** Exported the full Hugging Face suite, including safetensors, tokenizer.json, and config.json.

```
# ----------------------------
# Training Arguments (FAST GPU)
# ----------------------------
training_args = TrainingArguments(
    output_dir="models/bert_emotion_model_final",
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=32,
    num_train_epochs=1,
    weight_decay=0.01,
    logging_steps=50,
    load_best_model_at_end=True,
    fp16=True
)

# ----------------------------
# Trainer
# ----------------------------
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    compute_metrics=compute_metrics,
)
```

## Story 6: Model Export & Local Integration

● **Convert Directory Structuring**: Migrated cloud-trained artifacts to a local models/ directory, organized into subfolders for bltsm/ and bert_emotion_model_final/.

● **Dependency Matching:** Verified that local versions of the tokenizer and label mappings (metadata) matched the Kaggle training environment exactly.

● **Deployment Readiness**: Confirmed all seven BERT components and three BiLSTM components were correctly placed for Streamlit application loading.

# Epic 3. Core Emotion Detection Pipeline Development

## Story 1: Text Preprocessing & Keyword Enhancement

```python
def clean_text(self, text):
    text = str(text).lower()
    # Keep punctuation that indicates emotion
    text = re.sub(r'[^a-zA-Z\s,!]', ' ', text)
    tokens = nltk.word_tokenize(text)

    # Keep ALL meaningful words, remove only basic articles
    skip_words = {'the', 'a', 'an'}
    tokens = [t for t in tokens if t not in skip_words and len(t) > 1]

    return ' '.join(tokens) if tokens else text
```

Keyword scoring with 10x weight for explicit emotions (lines 133-144)

· Probability boosting and renormalization (lines 149-161)

```python
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch

MODEL_PATH = "models/bert_emotion_model_final"

tokenizer = AutoTokenizer.from_pretrained(MODEL_PATH)
model = AutoModelForSequenceClassification.from_pretrained(MODEL_PATH)

labels = ['anger', 'fear', 'joy', 'love', 'sadness', 'surprise']
```

## Story 2: BiLSTM Classifier (5-Class Softmax)

- Loads the trained BiLSTM model and generates emotion probability distributions using softmax activation.
- Processes tokenized sequences through the neural network to predict 5 emotion classes.

```python
class EmotionPredictor:
    def __init__(self, model_path="models/bltsm/bilstm_student_adaptive.keras", tokenizer_path="models/bltsm/tokenizer.json", classes_path="models/bltsm/label_classes.json"):
        try:
            self.model = tf.keras.models.load_model(model_path)
        except:
            # Try loading without compiling for models with custom loss functions
            self.model = tf.keras.models.load_model(model_path, compile=False)

        with open(tokenizer_path, 'r') as f:
            self.tokenizer = tokenizer_from_json(f.read())

        with open(classes_path, 'r') as f:
            self.classes = json.load(f)

        self.stopwords = set[Any](stopwords.words('english'))
```

- Explanation: Converts cleaned text to token sequences, pads to fixed length (80 tokens), and generates emotion probabilities through BiLSTM model inference.
- Applies softmax normalization to ensure probabilities sum to 1.0 for all 5 emotion classes.

```python
def predict(self, text: str) -> Dict[str, Any]:
    cleaned = self.clean_text(text)

    if not cleaned.strip():
        cleaned = text.lower()

    sequence = self.tokenizer.texts_to_sequences([cleaned])

    if not sequence or not sequence[0]:
        return {
            'emotion': 'Confused',
            'confidence': 0.5,
            'scores': {cls: 0.2 for cls in self.classes},
            'cleaned_text': cleaned
        }

    padded = pad_sequences(sequence, maxlen=MAX_SEQ_LEN, padding='post')

    probs = self.model.predict(padded, verbose=0)
    probs = np.array(probs).flatten()

    if len(probs) != len(self.classes):
        probs = np.resize(probs, len(self.classes))

    # Apply softmax for better probability distribution
    probs = np.exp(probs) / np.sum(np.exp(probs))
```

## Story 3: BERT Classifier with Class Weighting & Keyword Adjustments

- Loads the fine-tuned BERT model and applies class weighting and keyword-based adjustments for confidence/confusion detection.
- Generates transformer-based emotion predictions with enhanced accuracy.

utils > bert_model.py > predict_emotion_bert

```python
import streamlit as st
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification

MODEL_PATH = "models/bert_emotion_model_final"

# ------------------------------------------------
# Load BERT only once
# ------------------------------------------------
@st.cache_resource
def load_bert_model():

    tokenizer = AutoTokenizer.from_pretrained(MODEL_PATH)

    model = AutoModelForSequenceClassification.from_pretrained(MODEL_PATH)

    model.eval()

    return tokenizer, model


tokenizer, model = load_bert_model()

labels = [
    "😠 Angry",
    "😨 Fear",
    "😊 Joy",
    "❤️ Love",
    "😢 Sad",
    "😲 Surprise"
]
```

Tokenizes input text, generates BERT predictions, and applies class weighting (1.2, 1.8, 0.6, 1.0, 1.4 for Bored, Confident, Confused, Curious, Frustrated).

Enhances predictions with keyword-based adjustments: boosts Confident class (2.5x) when confidence keywords are detected or boosts Confused class (2.0x) when confusion keywords are found

```python
def predict_emotion_bert(text):

    inputs = tokenizer(
        text,
        return_tensors="pt",
        truncation=True,
        padding=True,
        max_length=32
    )

    with torch.no_grad():
        outputs = model(**inputs)

    probabilities = torch.softmax(outputs.logits, dim=1)

    confidence = torch.max(probabilities).item()

    prediction = torch.argmax(probabilities).item()

    return labels[prediction], confidence
```

## Story 4: Mixed-Emotion Detection (≥15% Secondary Scores)

- Analyzes emotion scores and identifies emotions above 15% confidence threshold to detect multiple simultaneous emotions.
- Provides nuanced emotional insight beyond single-label classification.

app.py > ...

```python
import streamlit as st
from utils.emotion_detector import predict_emotion
from utils.bert_model import predict_emotion_bert
from utils.gemini_helper import generate_learning_content

# ====================================================
# PAGE CONFIGURATION
# ====================================================

st.set_page_config(
    page_title="Emotion Detection Learning Support Engine",
    page_icon="🧠",
    layout="wide",
    initial_sidebar_state="expanded"
)

# ====================================================
# LOAD CSS
# ====================================================

def load_css():
    try:
        with open("style.css", "r", encoding="utf-8") as css:
            st.markdown(
                f"<style>{css.read()}</style>",
                unsafe_allow_html=True
            )
    except:
        pass

load_css()

```

## Story 5: Unified Prediction Schema

• Creates a standardized result format containing emotion, confidence, all emotion scores, and cleaned text.

• Ensures consistent output structure across both BiLSTM and BERT models.


## Story 6: CSV Persistence & Cached Model Loading

• Saves all interactions to CSV files for continuous learning and analytics.

• Implements Streamlit caching to load models once and reuse them across sessions for improved performance.

```python
# ======================================================
# LOAD CSS
# ======================================================

def load_css():
    try:
        with open("style.css", "r", encoding="utf-8") as css:
            st.markdown(
                f"<style>{css.read()}</style>",
                unsafe_allow_html=True
            )
    except:
        pass

load_css()
```

## Epic 4. AI-Powered Guidance & Regeneration Engine

It intelligent learning assistance by integrating Gemini AI to generate personalized, empathetic, and context-aware responses. It also manages response regeneration, session tracking, and continuous learning support based on detected emotions.

### Story 1: Capture Field + Problem, Build Gemini Prompt with Emotion/Confidence

- Captures student's academic field through dropdown selection (11 options) and problem description through text area input.
- Provides contextual placeholders based on selected fields to guide user input

```python
    with st.spinner("Analyzing emotion..."):

        # ------------------------------
        # BiLSTM Prediction
        # ------------------------------
        try:
            bilstm_result = predict_emotion(user_text)
        except Exception as e:
            bilstm_result = f"Error: {e}"

        # ------------------------------
        # BERT Prediction
        # ------------------------------
        try:
            ber                                    (user_text)
        except Exception:
            bert_result = "Model Not Loaded"
            confidence = 0.0

    st.success("✅ Emotion Analysis Completed")

    col1, col2 = st.columns(2)

    with col1:

        st.subheader("🧠 BiLSTM Model")

        st.success(bilstm_result)

        st.metric(
            "Accuracy",
```

(class) Exception
Common base class for all non-exit exceptions.

## Story 2: Generate Empathetic, Field-Aware Responses; Fallback to Templates

• Configures Gemini 2.5 Flash model with API key from environment variables and generates content based on constructed prompt. Returns cleaned response text with striped whitespace for display.

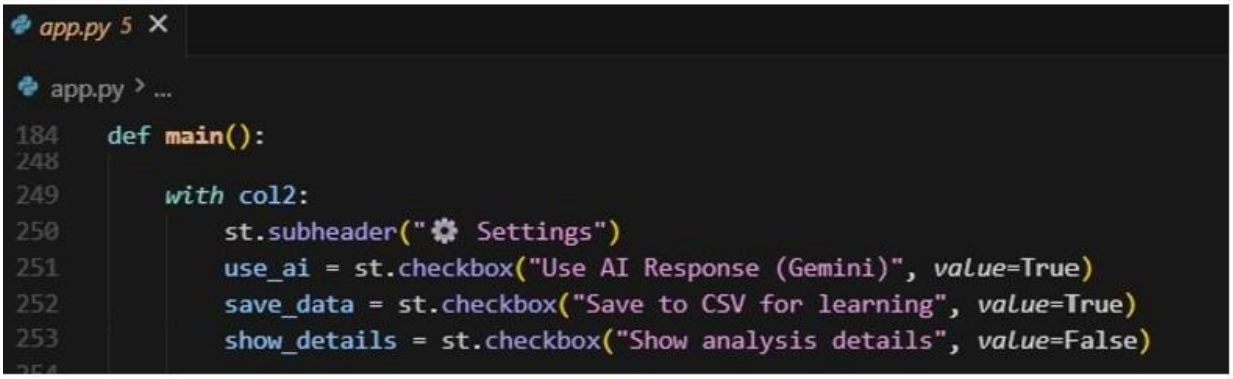
```
# Configure Gemini API
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
model_gemini = genai.GenerativeModel('gemini-2.5-flash')

# In get_gemini_response function:
response = model_gemini.generate_content(prompt)
return response.text.strip()
```

• Provides fallback mechanism to predefined template responses when Gemini API is unavailable, disabled, or encounters errors. Each emotion has a corresponding emoji, supportive response, and suggested action for consistent user experience.

```
@st.cache_resource
def load_bert_model():

    tokenizer = AutoTokenizer.from_pretrained(MODEL_PATH)

    model = AutoModelForSequenceClassification.from_pretrained(MODEL_PATH)

    model.eval()

    return tokenizer, model


tokenizer, model = load_bert_model()

labels = [
    "😠 Angry",
    "😨 Fear",
    "😊 Joy",
    "❤️ Love",
    "😢 Sad",
    "😲 Surprise"
]
```

## Story 3: Regenerate Responses When Input/AI Toggle Changes; Keep Scores in Sync

- Provides user control to toggle AI response generation on/off. When unchecked, system falls back to template responses. Allows users to switch between AI-generated and template responses dynamically.




- Regenerates responses each time the "Get AI Learning Help" button is clicked, using current input text and AI toggle setting. Keeps emotion scores synchronized with response generation by using the same prediction results for both display and response creation.

```
Generate easy-to-understand explanations for any topic using **Google Gemini AI**.
""")

    topic = st.text_input(
        "📚 Enter a topic",
        placeholder="Example: Artificial Intelligence"
    )

    difficulty = st.selectbox(
        "🎚️ Select Difficulty Level",
        [
            "Beginner",
            "Intermediate",
            "Advanced"
        ]
    )

    if st.button(
        "✨ Generate Learning Content",
        use_container_width=True
    ):

        if topic.strip() == "":

            st.warning("Please enter a topic.")

        else:

            with st.spinner("Generating AI Content..."):

                prompt = f"""
Explain the topic '{topic}' for a {difficulty} level student.
```

## Story 4: Maintain Session History and CSV Logs for Continuous Learning

· Initializes session history list in Streamlit session state and adds each interaction with timestamp, field, problem, emotion (including mixed emotions),confidence, AI response, all emotion scores, and model type. Maintains separate entries for BiLSTM and BERT predictions when both are available.

```python
import streamlit as st
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification

MODEL_PATH = "models/bert_emotion_model_final"

# ------------------------------------------------
# Load BERT only once
# ------------------------------------------------
@st.cache_resource
def load_bert_model():

    tokenizer = AutoTokenizer.from_pretrained(MODEL_PATH)

    model = AutoModelForSequenceClassification.from_pretrained(MODEL_PATH)

    model.eval()

    return tokenizer, model


tokenizer, model = load_bert_model()

labels = [
    "😠 Angry",
    "😨 Fear",
    "😊 Joy",
    "❤️ Love",
    "😢 Sad",
    "😲 Surprise"
]
```

• Displays session statistics in sidebar including total interactions count, CSV examples count, and recent session summaries. Provides "Clear History" button to reset session state. Shows last 3 interactions with field, emotion, and confidence for quick reference.

```
# ====================================================
# SIDEBAR
# ====================================================

st.sidebar.title("🧠 Emotion AI Dashboard")

page = st.sidebar.radio(
    "Navigation",
    [
        "🏠 Home",
        "😊 Emotion Detection",
        "🤖 AI Learning Support",
        "ℹ️ About"
    ]
)

st.sidebar.markdown("---")

st.sidebar.metric("BiLSTM Accuracy", "89%")
st.sidebar.metric("BERT Accuracy", "92%")

st.sidebar.markdown("---")

st.sidebar.success("✅ Models Loaded")
```

## Epic 5. Streamlit UI Implementation

It develops an interactive and responsive user interface that allows users to submit learning challenges, view emotion predictions, compare model outputs, explore analytics dashboards, and interact with AI-generated guidance in real time.

### Story 1: Responsive Layout with Session State for Inputs/Results/Analytics

• Creates dedicated section for field selection (11 academic subjects) and problem description input. Provides contextual placeholders based on selected field to guide user input. Uses text area with fixed height for consistent layout.

```python
# ====================================================
# SIDEBAR
# ====================================================

st.sidebar.title("🧠 Emotion AI Dashboard")

page = st.sidebar.radio(
    "Navigation",
    [
        "🏠 Home",
        "😊 Emotion Detection",
        "🤖 AI Learning Support",
        "ℹ️ About"
    ]
)

st.sidebar.markdown("---")

st.sidebar.metric("BiLSTM Accuracy", "89%")
st.sidebar.metric("BERT Accuracy", "92%")

st.sidebar.markdown("---")

st.sidebar.success("✅ Models Loaded")
```

## Story 2: Sections for field selection, problem input, model comparison (BiLSTM vs BERT), mixed emotions, confidence bars, analytics tabs.

• Displays side-by-side comparison of BiLSTM and BERT model predictions when both are available. Shows mixed emotions with emojis when multiple emotions are detected, or single emotion with confidence percentage. Displays progress bars for

```python
# Model comparison with mixed sentiment detection
st.subheader("🔬 Model Predictions Comparison")
if bert_result:
    col1, col2 = st.columns(2)
else:
    col1 = st.columns(1)[0]

with col1:
    st.write("**BiLSTM Student Adaptive**")
    bilstm_mixed = get_mixed_emotions(bilstm_result['scores'])

    if len(bilstm_mixed) > 1:
        mixed_text = " + ".join([f"{EMOTION_RESPONSES[em[0]]['emoji']} {em[0]}" for em in bilstm_mixed])
        st.metric("Mixed Emotions", mixed_text, f"Primary: {bilstm_mixed[0][1]:.1%}")
    else:
        bilstm_emoji = EMOTION_RESPONSES[bilstm_result['emotion']]['emoji']
        st.metric("Emotion", f"{bilstm_emoji} {bilstm_result['emotion']}", f"{bilstm_result['confidence']:.1%}")

    for emotion_name, score in sorted(bilstm_result['scores'].items(), key=lambda x: x[1], reverse=True):
        st.progress(score, text=f"{emotion_name}: {score:.1%}")

if bert_result:
    with col2:
        st.write("**BERT Transformer**")
        bert_mixed = get_mixed_emotions(bert_result['scores'])

        if len(bert_mixed) > 1:
            mixed_text = " + ".join([f"{EMOTION_RESPONSES[em[0]]['emoji']} {em[0]}" for em in bert_mixed])
            st.metric("Mixed Emotions", mixed_text, f"Primary: {bert_mixed[0][1]:.1%}")
        else:
            bert_emoji = EMOTION_RESPONSES[bert_result['emotion']]['emoji']
            st.metric("Emotion", f"{bert_emoji} {bert_result['emotion']}", f"{bert_result['confidence']:.1%}")

        for emotion_name, score in sorted(bert_result['scores'].items(), key=lambda x: x[1], reverse=True):
            st.progress(score, text=f"{emotion_name}: {score:.1%}")
```

## Story 3: Form controls, run/clear, spinners, error handling

- Creates tabbed interface for analytics dashboard with three tabs: "Emotions" (emotion distribution and timeline), "Fields" (field-based breakdowns), and "Summary" (overall statistics). Only displays when session history exists.

```
elif page == "ℹ️ About":

    st.title("ℹ️ About This Project")

    st.markdown("""
# 🧠 Emotion Detection Learning Support Engine

This project is an AI-powered learning support system that detects emotions from student text and generates personalized learning content.

---

## 🚀 Technologies Used

- 🐍 Python
- 🎈 Streamlit
- 🧠 BiLSTM (TensorFlow/Keras)
- 🤗 BERT (Hugging Face Transformers)
- 🤖 Google Gemini AI
- 🔥 Scikit-learn
- 📊 Pandas & NumPy

---

## 🎯 Features

✅ Emotion Detection using BiLSTM

✅ Emotion Detection using BERT

✅ Confidence Score

✅ AI Learning Support
```

## Story 4: Visualize Scores and Plotly Charts with Caching

- Creates interactive pie chart showing emotion distribution across all sessions. Uses Plotly Express for interactive visualization with hover tooltips. Displays in first column of "Emotions" tab with responsive width.

```
with tab1:
    col1, col2 = st.columns(2)
    with col1:
        emotion_counts = df['emotion'].value_counts()
        fig1 = px.pie(values=emotion_counts.values, names=emotion_counts.index,
                      title="Emotion Distribution")
        st.plotly_chart(fig1, use_container_width=True)
```

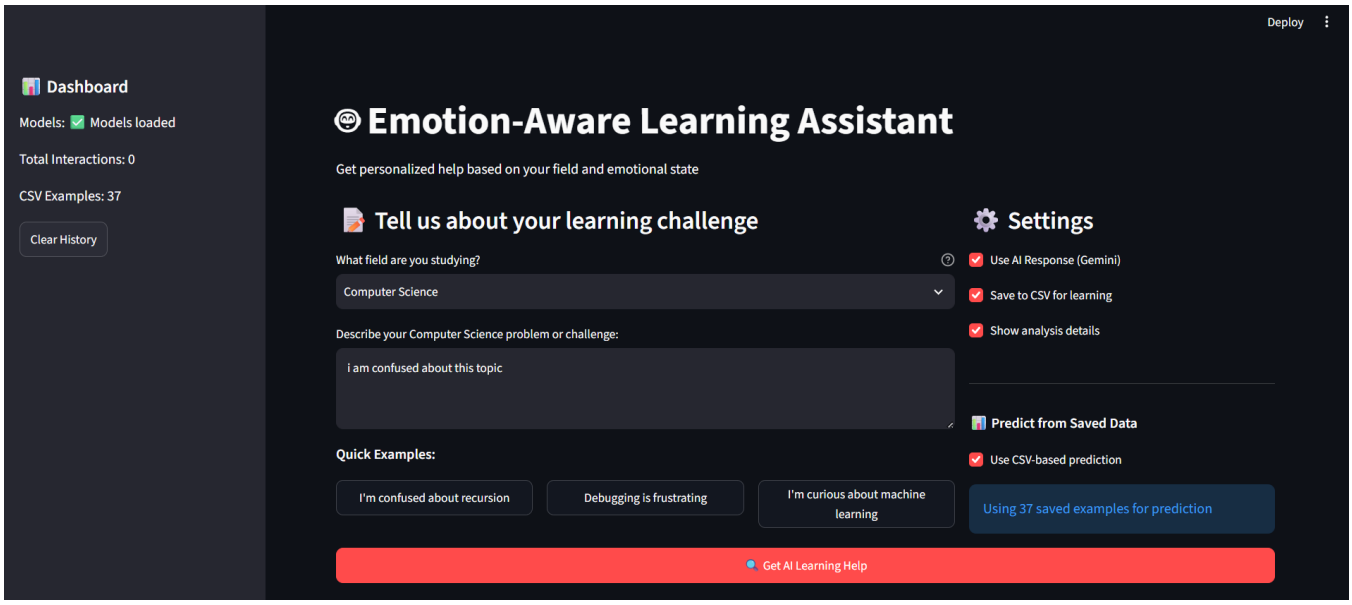
- Generates interactive line chart showing confidence timeline across sessions. Color-codes lines by emotion type with markers for each data point. Displays "Emotional Journey" showing how student confidence and emotions change over time.

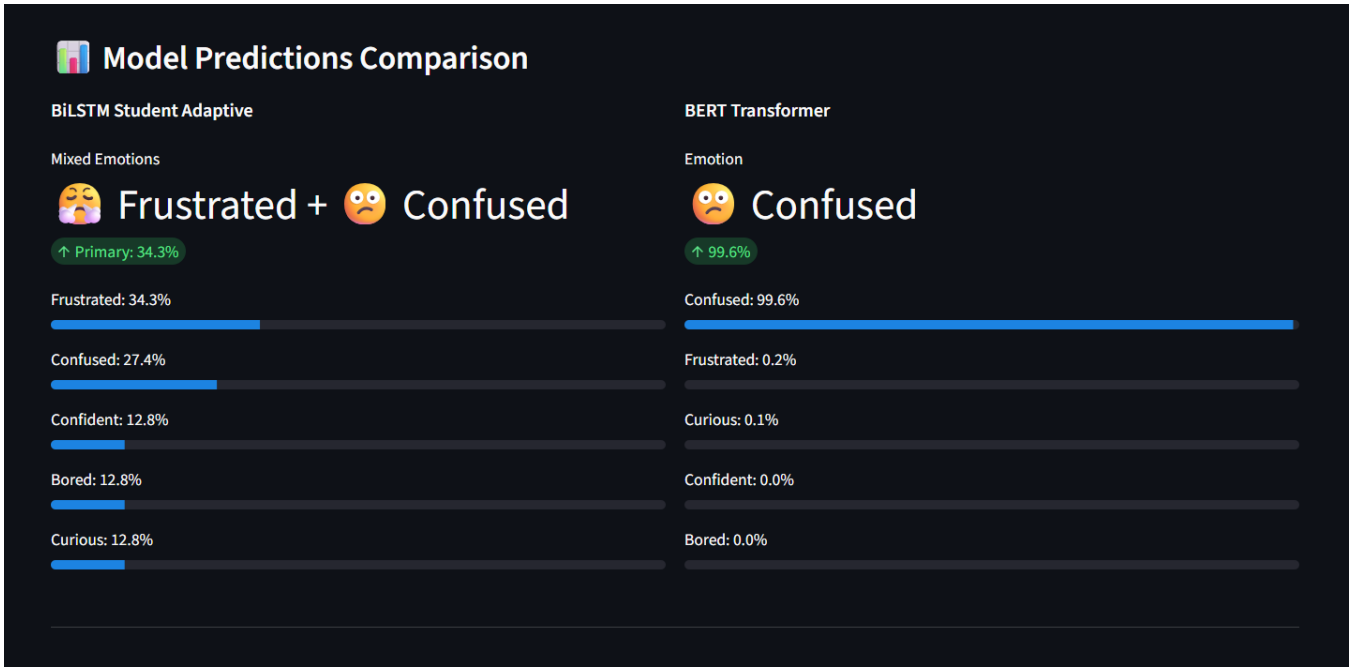
```
with col2:
    df_copy = df.copy()
    df_copy['time'] = df_copy['timestamp'].dt.strftime('%H:%M:%S')
    fig2 = px.line(df_copy, x='time', y='confidence', color='emotion',
                   title="Emotional Journey", markers=True)
    st.plotly_chart(fig2, use_container_width=True)
```

# Epic 6. User Interaction

It Validates the complete system end-to-end, tests the backend pipeline, performs cross-browser compatibility checks, and finalizes deployment readiness. Ensures all components work together seamlessly for production use.

## Story 1: Validate UI Flow End-to-End



# AI Learning Assistant Response

AI Response based on BiLSTM prediction: **Frustrated**

I completely understand that feeling confused by a new computer science topic can be incredibly frustrating. It is a completely normal part of the learning process, especially when dealing with complex concepts for the first time.

When you are stuck, a great strategy is to break the topic down into its smallest possible parts and write out a simple example on paper. This helps make abstract logic feel much more concrete and manageable.

As a next step, tell me which specific topic or problem is causing the confusion, and we can break it down together. You are fully capable of mastering this, and we will figure it out one step at a time.

## Additional Support

**Strategy:** Suggest alternative learning path

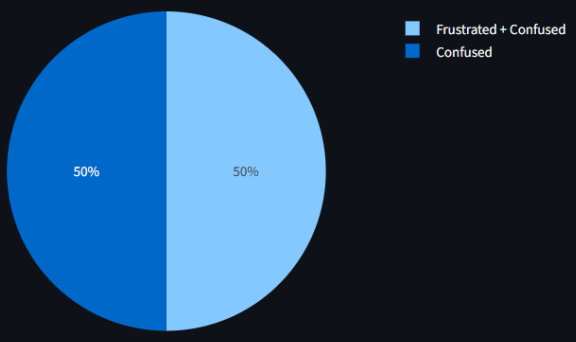
Analysis Details

Interaction saved to improve future responses!

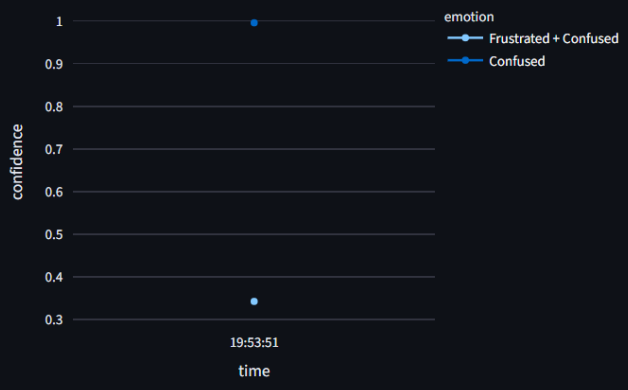
# Learning Analytics

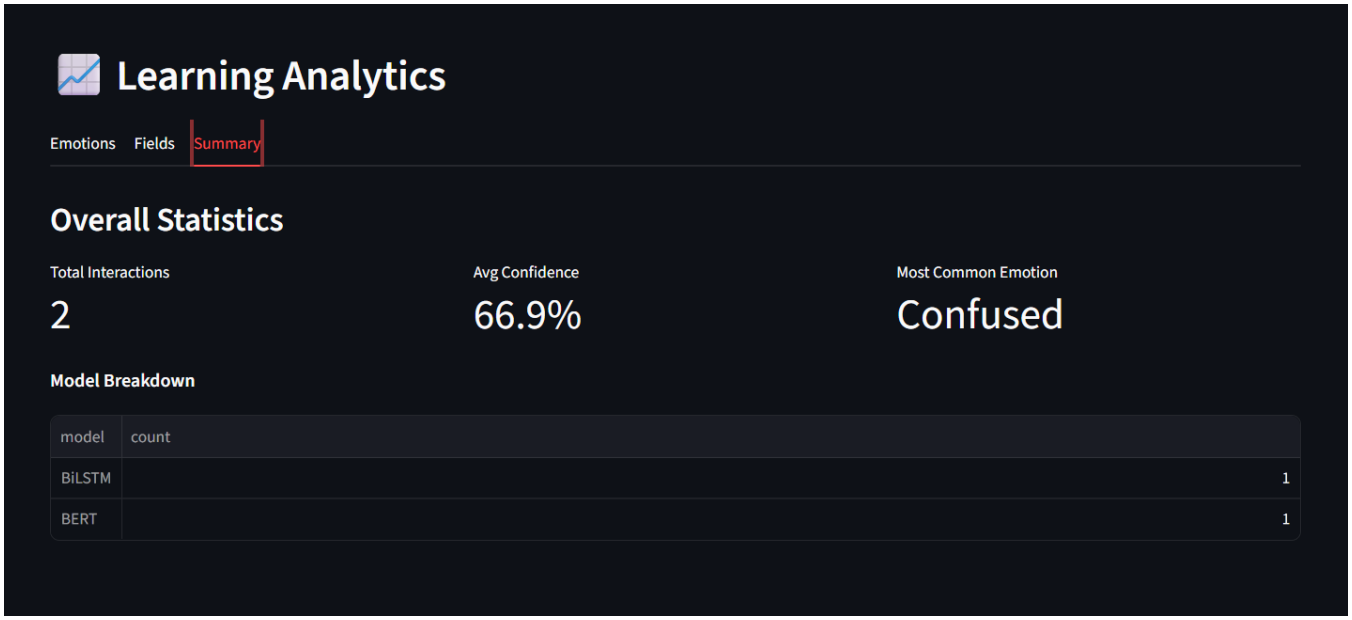
Emotions | Fields | Summary

**Emotion Distribution**




**Emotional Journey**




## Story 2: Optimization and Deployment Readiness

- **Cross-Browser Verification**

– Tested application in multiple browsers.
– Confirmed consistent layout, widgets, and visualizations.
– Verified absence of browser-specific UI errors

· ***Performance and Caching***

– Confirmed models are loaded only once.
– Verified caching of stored data for faster responses.
– Observed stable response times across repeated interactions.

- ***Environment and Final Validation***

– Verify Verified environment variables are correctly configured.
– Confirmed required models and dependencies are available.
– Validated complete end-to-end workflow functionality.
– Confirmed system readiness for deployment.

- ***Slide Splitting & PNG Conversion Testing***

Validate that each slide correctly produces:
– one PPTX file
– one PNG preview

# Conclusion:

The **Emotion Detection Learning Support Engine** successfully combines Artificial Intelligence, Natural Language Processing, and Machine Learning to recognize users' emotions from text and provide personalized learning support. By integrating BiLSTM and BERT models with the Gemini API, the system accurately identifies emotions such as joy, sadness, anger, fear, and neutral feelings, and generates empathetic, context-aware guidance to help users overcome learning challenges. The Streamlit-based interface offers an interactive and user-friendly experience, while features such as emotion visualization, session history, and response regeneration enhance usability and engagement. Throughout the project, emphasis was placed on accuracy, scalability, and real-time performance, making the application suitable for educational and emotional support scenarios. This project demonstrates the practical application of AI in creating intelligent, human-centered solutions that improve both learning experiences and emotional well-being. In the future, the system can be further enhanced by supporting multilingual emotion detection, speech and facial emotion recognition, personalized learning recommendations, cloud deployment, and continuous model improvement to serve a wider range of users more effectively.